Machine Learning with Applications 10 (2022) 100403

Contents lists available at ScienceDirect

Machine Learning with Applications

journal homepage: www.elsevier.com/locate/mlwa

# Improving text classification with transformers and layer normalization

Ben Rodrawangpai, Witawat Daungjaiboon *

*Underwriters Laboratories, 333 Pfingsten Road, Northbrook, IL 60062, United States*

ARTICLE INFO

*Keywords:*
Computational modeling
Machine learning
Natural language processing
Pattern classification
Semantics
Text analysis
Transformers

ABSTRACT

More than 25,000 injuries and 25 fatalities occur each year due to unstable furniture tip-over incidents. Classifying these furniture tip-over incidents is an essential task for understanding incident patterns and building safer products. For example, this classification can help standards development organizations (SDOs) and policy makers discover hidden insights, which can be used to develop standards and regulations that help improve furniture and make homes safer. Since 2000, the U.S. Consumer Product Safety Commission (CPSC) has published data related to consumer product injuries. The amount of data has grown rapidly, and the process of manually reviewing and classifying individual incidents has correspondingly become very resource intensive. This paper proposes an improved method that employs a combination of natural language processing (NLP) techniques and machine learning (ML) algorithms to classify textual data. Machine learning models can help reduce time and effort by streamlining incident narrative classification for determining whether incidents are related to furniture tip-overs. Challenges often presented by real-world data sets (such as the CPSC data used in our experiment) include imbalanced target classes and narratives requiring domain knowledge, since the data sets contain abbreviations and jargon. Using out-of-the-box, default classification models such as bidirectional encoder representations from transformers (BERT) might not yield adequate results. Our proposed method adds layer normalization and dropout layers to a transformer-based language model, which achieves better classification results than using a transformer-based language alone with imbalanced classes. We carefully measure the impact of hidden layers in order to fine-tune the model.

## 1. Introduction

In its 2020 report on furniture tip-over injuries and fatalities in the U.S., the CPSC estimated that these incidents result in a yearly average of 25,000 injuries that require treatment in a hospital emergency department. The analysis also identified that 571 deaths occurred from furniture tip-over incidents between 2000 and 2019, with children under 18 years old making up a majority of the fatalities (82%) (Suchy, 2020). Government agencies, including the CPSC, collect data to identify risks and hazards to people, then act to address unreasonable risks to citizens. For almost 50 years, the CPSC has operated the National Electronic Injury Surveillance System (NEISS) to collect data on consumer product-related injuries occurring in the United States. NEISS relies on a nationally representative sample of hospitals in the U.S. to report patient information for every emergency department visit associated with a consumer product. The total number of product-related hospital emergency department visits across the country can be estimated from the data in NEISS (CPSC, 2020). The NEISS data set now includes more than 7.5 million records from 2000 through 2021.

Classifying incidents from a hospital emergency department and exploring emerging risks may require subject-matter experts to analyze millions of records, which is a time-intensive task. Leveraging the use of machine learning algorithms and natural language processing techniques can help reduce human efforts and increase efficiency for this analysis, while still maintaining or improving accuracy and precision. This reduced effort can assist manufacturers in the design of safer products, as well as government agencies in the identification of characteristics from unsafe consumer products that should be recalled from the market. Moreover, standard development organizations can benefit by adopting this application to streamline analysis, inform the development process and thereby improve appropriate safety standards.

A tip-over incident occurs when a piece of furniture falls on an individual. These incidents can be caused by poorly designed or manufactured furniture, particularly when combined with certain behaviors, such as opening multiple drawers or children climbing on the furniture. Dressers, bookcases and other tall pieces of furniture are more likely to fall because of their instability. In order to address furniture instability

The code (and data) in this article has been certified as Reproducible by Code Ocean: (https://codeocean.com/). More information on the Reproducibility Badge Initiative is available at https://www.elsevier.com/physical-sciences-and-engineering/computer-science/journals.

* Corresponding author.
*E-mail addresses:* ben.rodrawangpai@ul.org (B. Rodrawangpai), witawat.daungjaiboon@ul.org (W. Daungjaiboon).

https://doi.org/10.1016/j.mlwa.2022.100403
Received 1 March 2022; Received in revised form 12 August 2022; Accepted 12 August 2022
Available online 19 August 2022
2666-8270/© 2022 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

**Table 1**
Narrative text is presented as it appears in NEISS with our manual classification labels.

| Narrative | Label |
|---|---|
| 27 yof dresser fell over on her toe while unpacking. dx foot fx | tip-over |
| 4 yo male fell into dresser. dx chin laceration | non-tip-over |

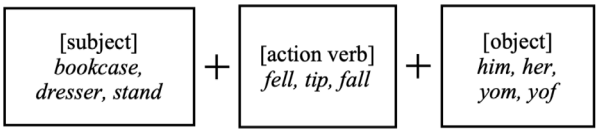



**Fig. 1.** A pattern of the tip-over incident narrative.

and the resulting injuries and deaths, the furniture industry has worked with standards development organizations to develop construction and testing standards for these products. However, furniture instability has continued to cause a significant number of injuries, with no identifiable trend in the number of emergency department treated injuries between 2010 and 2019 (Suchy, 2020).

Lu et al. (2021), stated that NEISS incident narratives do not capture many incident details and appear to be inconsistent. The inconsistency of data may cause improper classification of incident injuries. Furniture tip-over classification is a form of binary classification used to determine whether an incident narrative is a tip-over or non-tip-over incident. The incident classification task is time-consuming and labor-intensive. Additionally, classifying tip-over incidents is challenging because incident narratives are relatively short and filled with medical jargon and abbreviations. A change in word order often results in a change of meaning.

Table 1 shows examples of incident narratives from NEISS (CPSC, 2020). Both incidents are similar in term of words such as "dresser" and "fell". However, the subject of the first sentence is a dresser that falls over onto a woman — a tip-over incident. The subject of the second sentence is child that fell into a dresser — not a tip-over incident. This example also typifies the use of abbreviations: yof indicating "year-old female", dx for "diagnosis" and fx for "fracture".

In an unpublished 2017 study by Underwriters Laboratories, keyword analysis techniques and expert judgment were used to classify tip-over incidents. Fig. 1 shows an example of the defined pattern of tip-over incidents. If incident narratives were matched with the defined pattern, the tip-over incident label was assigned to the incident. An incident narrative that was not matched with the defined patterns, but contained selected keywords, required a domain expert to review and classify the incident manually. This method was resource intensive, as many events required manual review. As the data grew, more time was required to manually review individual incidents. Therefore, the application of text classification using machine learning algorithms lends itself very well to overcoming manual efforts in Big Data era (Aggarwal & Zhai, 2012).

A machine learning algorithm using Term Frequency–Inverse Document Frequency (TF–IDF) and a Naïve Bayes classifier (Ramos, 2003; Rish, 2001) was also applied to the data to classify tip-over events in the 2017 study. However, the model did not perform well because TF–IDF could not capture the position of words and semantic differences in incident narratives (Kowsari et al., 2019).

With semantic limitations of TF–IDF, we hypothesized that a new language model representation, BERT, could overcome and perform better than TF–IDF in our application. The application of these techniques to the analysis of incident narratives could help reduce the time and effort associated with studies to identify patterns of injury and the products that cause them. Given the challenges associated with the classification of tip-over incidents, we further hypothesized that the methods may be applied to other incident classification problems.

In this paper, we propose a new text classification model by adding layer normalization, followed by Dropout layers to the pre-trained transformer model. We also benchmark the proposed model against the transformer-based text classification models including BERT, RoBERTa, DeBERTa, ALBERT, DistilBERT, and MPNet.

## 2. Related work

There are multiple research reports that show how the variation of BERT models have evolved and improved in performance over time. In regards to the implementation of natural language processing applications, it is proven that applying transfer learning (which implements pre-trained language models) and adjusting multiple parameters for a specific task can help predict the likelihood of furniture tip-overs from incident narratives.

### 2.1. Transformers

A transformer is a neural network that transduces input data into vector representations using self-attention layers with encoder–decoder structure (Vaswani et al., 2017). Development of the transformer model was based on multiple base models, and BERT is one of the most well-known models using transformer architecture. BERT is a powerful language representation model. In a 2019 study, Devlin et al. proved that BERT, a bidirectional pre-trained language model, outperformed Open AI GPT, a unidirectional language model, in all NLP tasks by taking context from both directions into account (Devlin et al., 2019). In the study, the authors state that there are two steps involved in BERT implementation: pre-training and fine-tuning. They found that fine-tuning the BERT model is quite inexpensive when compared with pre-training. The use of the BERT framework is also straightforward, since it has a unified architecture in a variety of tasks due to simply using inputs and outputs for a specific task. There have been extensive improvements in BERT models, including Lightweight BERT (DistilBERT) and Robustly Optimized BERT (RoBERTa).

A robustly optimized BERT pretraining approach was implemented by Liu, et al. to improve the BERT base model by using dynamic masking, full sentence without next sentence prediction (NSP) loss, large mini-batches, and larger Byte-Pair Encoding (BPE) techniques. By combining these implementation techniques, Liu, et al. proved that the RoBERTa language model established significant performance improvement from the BERT base model (2% to 4%) in most NLP downstream tasks (Liu et al., 2019). Improving performance of pre-trained language models, especially BERT model architecture, for a specific task is computationally expensive due to increases in the size of the models during learning natural language representation processes. In order to make BERT more scalable and reduce computational power during training process, Lan, et al. extend the BERT model by applying two techniques: decomposing embedding parameters into two smaller matrices, and sharing parameters across multiple layers. The proposed model is called a Lite BERT (ALBERT). While ALBERT has fewer parameters, it still yields better results than the BERT-large model across General Language Understanding Evaluation (GLUE), Large-Scale ReAding Comprehension Dataset from Examinations (RACE), and Stanford Question Answering Dataset (SQuAD) benchmarks (Lan et al., 2019).

In DistilBERT (Sanh et al., 2020), Sanh et al. developed a method to reduce the computational expenses of training a language representation model by using a smaller, pre-trained BERT model for general purposes while still retaining BERT's performance and its capability of fine-tuning for specific tasks. They found that their distilled version of BERT computed 60% faster with 40% fewer parameters than the original BERT and RoBERTa models, while still preserving 97% of language understanding capabilities.

In 2020, a new model called the Masked and Permuted Pre-Training Model (MPNet) was developed in a collaboration between Microsoft and researchers from Nanjing University of Science and Technology.

The developers adopted the advantages of using masked language modeling (MLM) and permuted language modeling (PLM) methods from BERT and XLNet respectively (MLM to efficiently leverage bidirectional context of masked tokens, PLM to capture dependencies among the predicted tokens). MLM helped to better extract information from full sentences during pre-training and fine-tuning processes, which compensated input consistency, but PLM did not know the position of the full sentence. However, PLM prevented the assumption that masked tokens are independent by factorizing predicted tokens. The MPnet was then trained on large text corpora with a size exceeding 160 GB and fine-tuned for multiple downstream NLP tasks. The model outperformed previous state-of-the-art, pre-trained language models (BERT, XLNet, RoBERTa, e.g.) on multiple downstream NLP tasks from using strengths from XLNet and BERT (Song et al., 2020).

Microsoft researchers proposed Decoding-Enhanced BERT With Disentangled Attention (DeBERTa) in 2020 by incorporating two new novel techniques to improve BERT and RoBERTa. These two techniques were a disentangled attention mechanism and an enhanced mask decoder. The techniques made DeBERTa outperform the RoBERTa model by 0.9% to 3.6% on multiple downstream tasks, the Multi-Genre Natural Language Inference Corpus (MNLI), SQuaD, and RACE, while only being trained on half of training data sets. DeBERTa's performance was also on top of the SuperGLUE leaderboard in 2021 with a 0.5% improvement from the human baseline (He et al., 2020).

### 2.2. Fine tuning pre-trained model

Fine tuning a pre-trained language model has proven its performance when data is large enough in previous works. Multiple studies found this technique performs well in text classification tasks. Lee and Hsiang proposed the use of the BERT language model as a base model and fine-tuned the BERT model to categorize U.S. patents (Lee & Hsiang, 2019).

In their experiment, Lee and Hsiang found that the BERT base model outperformed Li et al.'s DeepPatent (a Deep learning algorithm with word embeddings) (Li et al., 2018). Their fine-tuned BERT model for patents achieved a F1 score of 66.83%, precision score of 84.26%, and recall score of 55.38%. They also found this technique still performed well when they used different test data sets.

### 2.3. Layer normalization

Layer normalization (LayerNorm) has been used to shorten training processes and improve model performance. In a study by Lei Ba et al. study (Ba et al., 2016), it was proven that applying layer normalization in deep neural networks can help training models converge to achieve better results faster and increase model performance in multiple tasks. Fig. 2 shows the calculation of layer normalization. Layer normalization is dot production represented as $\odot$ of centered and scaled of inputs as shown in the formula below (Xu et al., 2019).

$$h = g \odot N(x) + b$$

$$N(x) = \frac{x - \mu}{\alpha}$$

$$\mu = \frac{1}{H}\sum_{i=1}^{H} X_i$$

$$\alpha = \sqrt{\frac{1}{H}\sum_{i=1}^{H}(x_i - \mu)^2}$$

Where:

$h$ = Output from Layer normalization

$x$ = Vector representation of inputs

$H$ = Number of inputs

$\mu$ = Mean of inputs

$\alpha$ = Standard deviation of inputs

$b$ = bias

$g$ = gain

**Fig. 2.** The calculation of layer normalization.

**Table 2**
Product Codes included in the dataset.

| Product Code | Description |
|---|---|
| 604 | Desks, dressers, chests, bureaus, or buffets |
| 4013 | Other furniture |
| 4014 | Furniture, not specified |

### 2.4. Dropout

Deep neural networks are complex, consisting of multiple, non-linear hidden layers, yet they are powerful machine learning algorithms that can yield good prediction results in various domains. The complicated nature of deep neural networks makes tuning parameters and detecting overfitting problems very challenging. There are multiple regularization techniques that can help to overcome overfitting problems. Dropout is one of the regularization techniques that prevents overfitting in model predictions. Srivastava et al. proved in a 2014 study that applying Dropout in deep neural networks can obtain better prediction results and significantly reduce overfitting issues, when compared to other regularization techniques (Srivastava et al., 2014). The study also shows great improvement in prediction performance of the model on multiple machine learning tasks including computer vision, speech recognition, text classification, and computational biology on varying benchmark data sets. Dropout randomly removes units from individual training batches (Srivastava et al., 2014). Doing so prevents the chance of a unit depending on other units, and it also makes each one learn a task on its own, which helps the models become more robust.

The studies mentioned above inspired us and helped us tailor our proposed model to improve efficiency and reduce human efforts in classifying tip-over incidents.

## 3. Methodology

### 3.1. Data

Narratives from CPSC's NEISS database from 2010 to 2015 were used as an input for a machine learning algorithm to classify whether an incident was a tip-over incident or not (CPSC, 2020). Target labels (tip over and non-tip over) were manually assigned based on context in incident narratives during our 2017 furniture tip-over analysis project by using keyword analysis and expert judgment. Table 2 shows product categories that were included in the dataset (CPSC, 2019).

Our target classes, tip-over and non-tip-over, are imbalanced. The classified data contains 1123 tip-over incidents and 7470 non-tip-over incidents. Those who are not domain knowledgeable may find it very challenging to understand the details of each incident due to abbreviations in incident narratives. We split the dataset into random training,

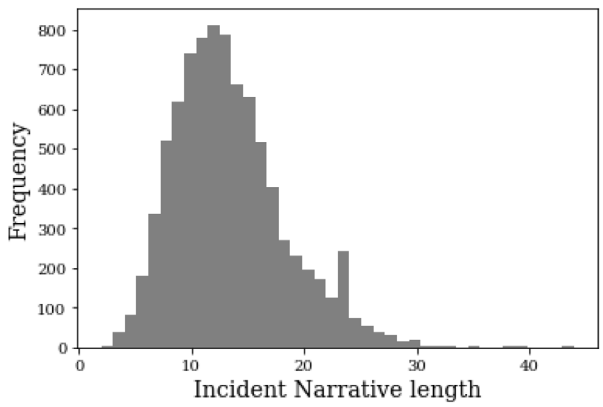



**Fig. 3.** The histogram of incident narrative length.

validation, and test sets. Training and validation data sets were used in training language models for our furniture tip-over classification task, and the test sets acted as unseen data to benchmark performance of our proposed model with different base models. The portion of data in training, validation, and test sets are 6874 (80%), 860 (10%), and 859 (10%) records respectively. Fig. 3 displays the distribution of incident narrative length, measured in words. The mean and median of incident length are 13.44 and 13.00 respectively. The longest incident length is 44 words.

After reviewing the distribution of incident narrative length, we then used a pre-trained tokenizer on the incident narratives and chose the maximum length of 32 for token size. Either truncation or padding was applied to ensure the same length of a token size.

### 3.2. Proposed model

We evaluated our proposed model by benchmarking against BERT model variants to determine the most useful combination of techniques for the task of classifying furniture injury incident narratives into tip-over and non-tip-over categories.

#### 3.2.1. Model architecture

In this work, our proposed model architecture is based on a pre-trained RoBERTa model. We modified a RoBERTa classification model by adding a layer normalization (LayerNorm) which enables smoother gradients, faster training, and better generalization accuracy (Xu et al., 2019), followed by the Dropout layer, which helps prevent overfitting, and a linear layer for classification on top of the transformer-based language model. Fig. 4 shows the architecture of our proposed model: RoBERTa with Layer Normalization, Dropout and classifier layers.

The transfer learning technique was also adopted to leverage a pretrained model to a specific domain (injury narratives from NEISS). The pretrained language models, BERT, RoBERTa, DeBERTa, ALBERT, DistilBERT, and MPNet were used to preprocess incident texts and to fine tune the pretrained models for text classification tasks.

Sentence narratives were preprocessed to convert texts into proper input formats for a specific model. All texts were split into a set of words, also known as "tokens". Special tokens were added to separate words based on types of languages models. Long sentences were truncated to be compatible with the maximum length of a model, and short sentences were padded to match the maximum length in each batch. Truncating and padding were used to keep each input at a consistent length for entire input batches that were compatible with a language model. Lastly, as illustrated in Fig. 4, all text tokens were encoded into input ID numbers, which represent individual separated words.

We also selected the default classification models, BERT, RoBERTa, DeBERTa, ALBERT, DistilBERT, and MPNet, from Transformer (Wolf et al., 2020) to compare with our proposed model, as indicated in

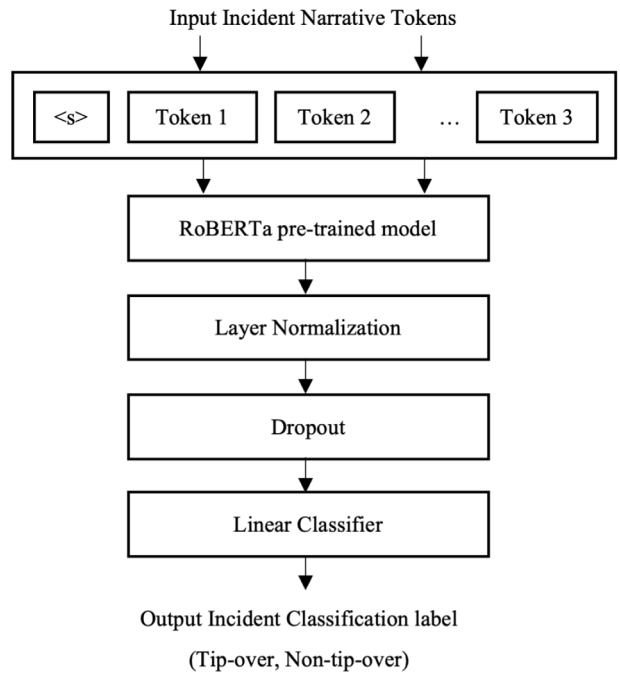



**Fig. 4.** The architecture of our proposed model: RoBERTa with Layer Normalization and Dropout classification model.

**Table 3**
Complexity of different layer types.

| Layer type | Number of layers | Complexity per layer |
|---|---|---|
| Attention layer | 12 | $O(n^2 \cdot d)$ |
| Normalization layer | 1 | $O(n \cdot d)$ |
| Dropout layer | 1 | $O(n \cdot d)$ |
| Classifier layer | 1 | $O(n \cdot d)$ |

Fig. 5. To fine-tune the process, all the models were trained on the same parameters for performance comparison. They were trained with the batch size of 100. Adam (Kingma & Ba, 2017) was used for optimization of the model with the learning rate of 1e-05. We used Transformers from HuggingFace (Wolf et al., 2020) as a framework and Python language for the implementation of machine learning models.

#### 3.2.2. Computational complexity

Transformers consist of multiple self-attention layers. Vaswani et al. (2017) stated each self-attention layer has complexity equal to $O(n^2 \cdot d)$ where $n$ is the sequence length and $d$ is the dimension of language representation. Our proposed model included a stack of 12 attention layers and one normalization, one Dropout and one classifier layer as indicated in Table 3. Normalization, Dropout and classifier layers are considered as linear operations therefore their individual complexity is equal to $O(n \cdot d)$.

Therefore, our proposed model's estimated complexity is calculated as shown below.

$$12O(n^2 \cdot d) + 3O(n \cdot d)$$

Computational complexity was only one of the aspects to identify performance of the proposed model, but we also took classification performance and time inference on the same computing settings into account for models' performance in this paper.

#### 3.2.3. Classification performance

Classification accuracy is the percentage of correctly predicted incidents to the total incidents.

$$accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

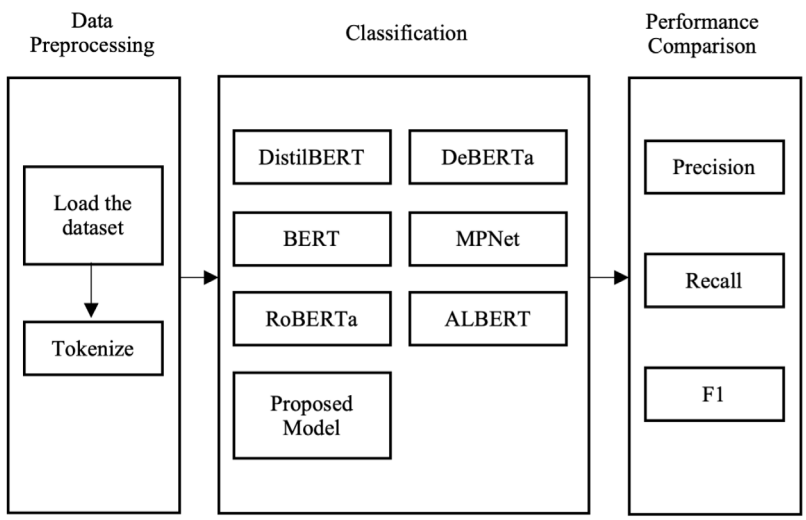



**Fig. 5.** Design approach for experiment and performance evaluation.

Where:

TP (True Positive) = Number of correctly predicted as tip-over incidents

TN (True Negative) = Number of correctly predicted as non-tip-over incidents

FP (False Positive) = Number of incorrectly predicted as tip-over incidents

FN (False Negative) = Number of incorrectly predicted as non-tip-over incidents

Using classification accuracy to assess model performance alone is not ideal when it comes to unequal distribution of target classes in the dataset. Classification accuracy may be biased because it tends to focus on the majority class while ignoring the minority class or treating it as noise.

Therefore, additional metrics should be used to assess each model's performance on the test set for the furniture tip-over classification task. Precision, recall and F1 scores generated by the Python package Scikit-learn (Buitinck et al., 2013; Pedregosa et al., 2011) were used to achieve fair model assessment on such an imbalanced data set. Precision measures the correctly classified positive cases from all the predicted positive cases. Recall is used to quantify the number of correctly classified as positive cases made out of all positive cases in the dataset. F1 score is the harmonic mean of Precision and Recall; it combines both measurements into a single measure.

$$Precision(P) = \frac{TP}{TP + FP}$$

$$Recall(R) = \frac{TP}{TP + FN}$$

$$F1Score = 2 * \left(\frac{R * P}{R + P}\right)$$

Macro average and weighted average were used to evaluate overall performance for each model. The macro average score does not take proportion of each class into account. The weighted average score, however, is calculated by determining weighting for each class individually based on actual numbers of true labels in each class. Since our data set appears to be imbalanced, using only macro average and weighted average scores may obscure how well models perform on each class individually. We therefore assessed each model on a class-wise level to see how well they predict individual classes.

## 4. Results

### 4.1. Learning curve

Learning curve was used to evaluate learning behavior on both training and validation sets of our proposed algorithm and other base language models. Plots in Fig. 6 shows learning error (losses) against epoch (the number of times each classifier learns an entire data set). Early stopping was used to halt the training and the best epoch number was applied to each text classification model.

Based on Fig. 6, the learning curve of our proposed model, RoBERTa with a normalization layer and Dropout, on both training and validation sets, descends within only one epoch. The curves continue to decrease and remain stable after four epochs with consistent distance between training and validation sets, indicating that it is a good fit.

### 4.2. Precision–recall (PR) curves

The calculation of area under the curves, receiver operating characteristic (ROC) and precision–recall curve (PRC), are often used as metrics to assess performance of classifiers when it comes to binary classification problems. PRC was used to see how well our algorithms performed on both classes. ROC is frequently optimistic because it only accounts for true positive and false positive rates — indicating whether predictive results are correct. Since our target classes were severely imbalanced, precision and recall were considered to better assess predictive performance of the models.

Area under the precision–recall curve (AUPRC) were reported in Fig. 7 to show how well each classifier performed for our furniture tip-over problems. Our proposed model obtained the most area (AUC= 0.95) compared to other transformer-based classification models. Applying a layer normalization and Dropout to a transformer-based language model showed significant improvement from a basic model (RoBERTa) by 11%.

### 4.3. Benchmarking

Each selected model was run to predict a tip-over classification task 10 times, and a performance model of each round was obtained for statistical analysis. Different models were evaluated using precision, recall, and F1 score to see how effectively each model classified our target class. Each table in this section represents our model performance measures on test data that we prepared in steps earlier, divided by target classes, tip-over and non-tip-over.

The performance of macro averaging scores among the models is reported in Table 4. Our proposed model was superior among default DistilBERT, BERT, RoBERTa, DeBERTa, ALBERT, and MPNet models, and gained a 4% to 7% improvement on precision, recall, and F1 scores.

The standard deviation of each model was reported in Tables 6, 7 and 9. With very small standard deviation values, our proposed model

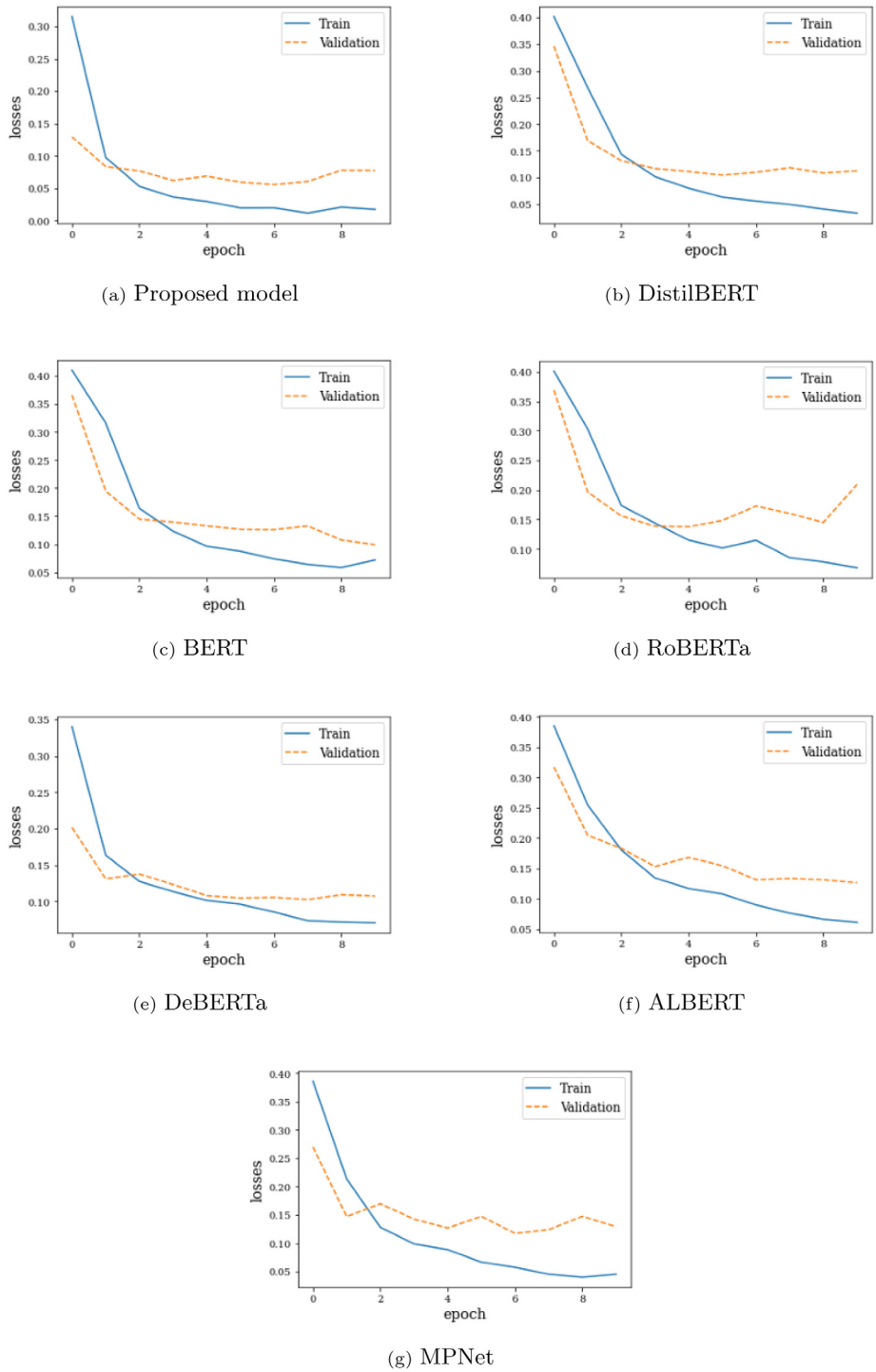
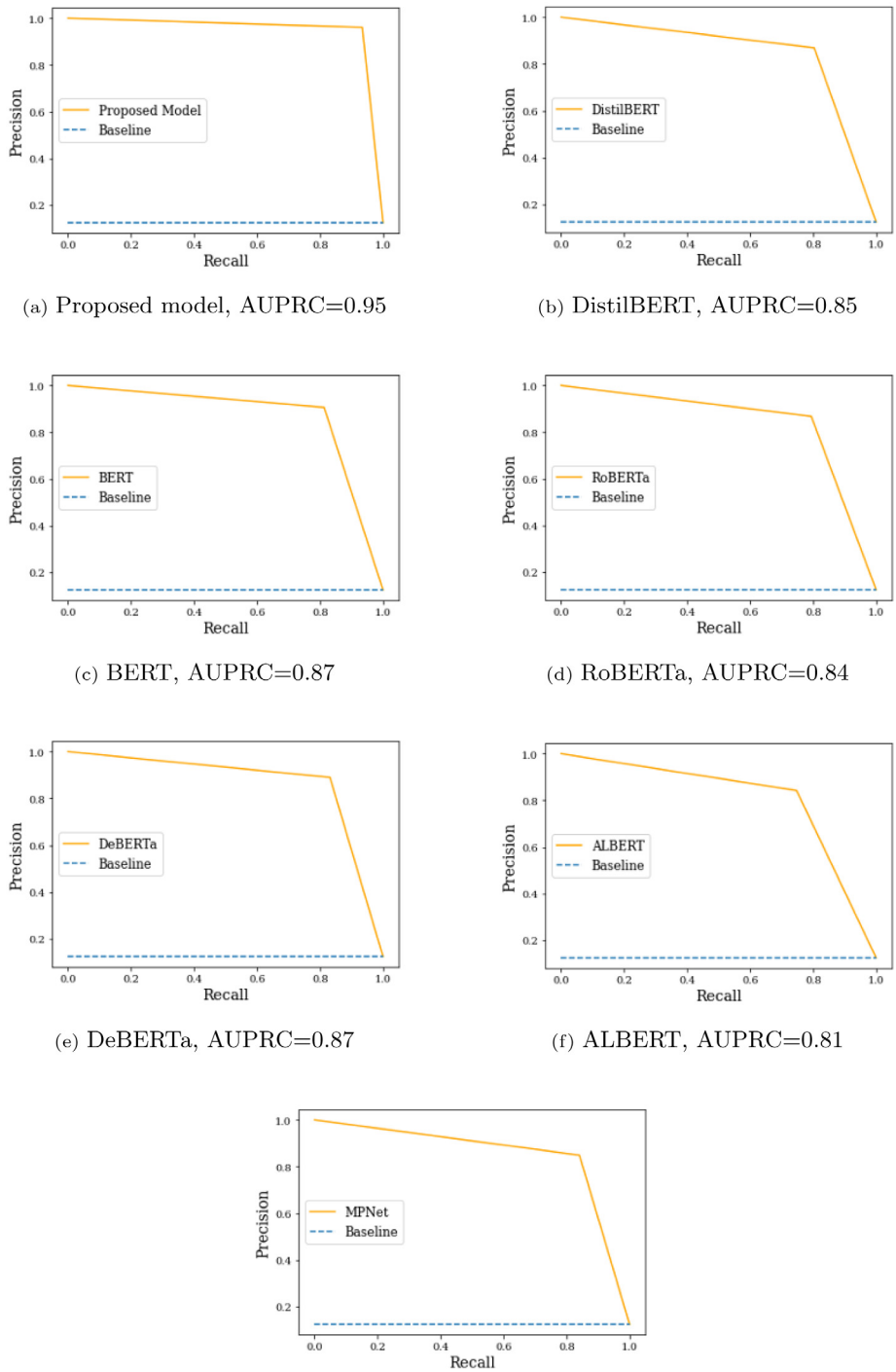



**Fig. 6.** The learning curves comparing training and validation sets (loss vs number of epochs) of text classification models.

for the tip-over classification task could still retain the performance of BERT models across all metrics.

Since the dataset is imbalanced, we used the F1 score to evaluate and compare the balance of the models' ability to recognize both tip-over and non-tip-over classes. Additionally, we measured the performance of both tip- and non-tip classes separately, instead of using macro averaging scores alone. As can be observed from Table 8 for granular class-wise performance, our proposed model showed a significant improvement in the accuracy for the minority class (tip-over incidents). The proposed model achieved an F1 score of 0.93, which is 8% to 13% higher than using out-of-the-box pre-trained models.

In addition to the models' performance results, Freidman's test and post-hoc Wilcoxon signed-rank test with Holm's alpha correction (5%) were performed on test data results acquired from making prediction each state-of-art classifications 10 times. Friedman's test was used to determine that our proposed model performs statistically better than other basic models. A critical difference diagram (Fawaz et al., 2019) was used to visualize results from both statistic tests. Multiple models




(a) Proposed model, AUPRC=0.95

(b) DistilBERT, AUPRC=0.85

(c) BERT, AUPRC=0.87

(d) RoBERTa, AUPRC=0.84

(e) DeBERTa, AUPRC=0.87

(f) ALBERT, AUPRC=0.81

(g) MPNet, AUPRC=0.85

**Fig. 7.** Precision–recall (PR) curves of text classification models.

were placed on the axis of the diagram based on their average ranks, with the lowest ranking models on the left and the best ranking models on the right. Thick horizontal lines between each data point indicated that models were not significantly different.

Our proposed model, RoBERTa with a layer normalization and Dropout outperformed other models. Having done statistical tests on models' performance, our proposed model was statistically superior among other out-of-the-box languages models. Our customized model is shown on the far right of the critical difference diagram without a thick horizontal line in Fig. 8.

The proposed model also yielded the best results among other language models in the minority class (tip-over incidents). Results from the proposed model were still placed on the right of the critical difference diagram of tip-over class F1 scores as shown in Fig. 9.

Average computational time consumption of each model was also reported in Table 5. Our proposed model, RoBERTa with layer normalization and Dropout, spent the same inference time as other BERT family models: BERT, RoBERTa, ALBERT, and MPNet. The proposed model with additional customized layers still preserved the performance of BERT models. The proposed model proved that adding normalization made the basic model yield better results on a specific task. The critical

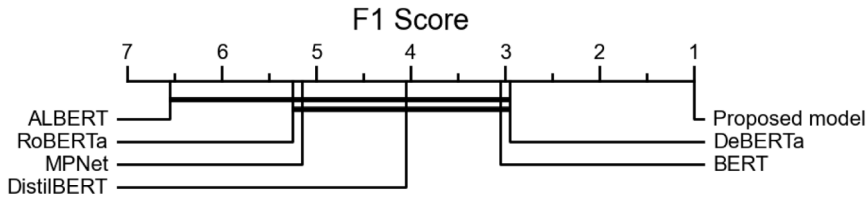



**Fig. 8.** Critical diagram of macro averaging F1 scores.

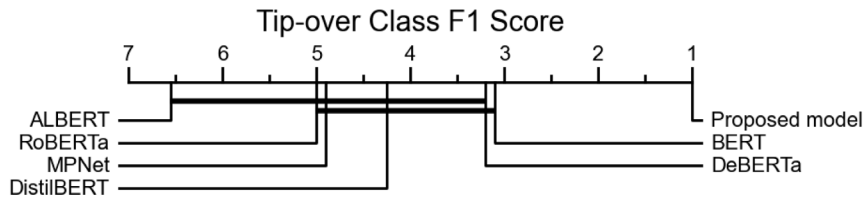



**Fig. 9.** Critical diagram for F1 scores of minority class (tip-over class).

**Table 4**
Classification report comparison: macro and weighted scores.

| Model | Macro | | | Weighted | | |
|---|---|---|---|---|---|---|
| | Precision | Recall | F1 | Precision | Recall | F1 |
| Proposed model | 0.96 | 0.96 | 0.96 | 0.98 | 0.98 | 0.98 |
| DeBERTa | 0.91 | 0.92 | 0.92 | 0.96 | 0.96 | 0.96 |
| BERT | 0.93 | 0.90 | 0.91 | 0.96 | 0.97 | 0.96 |
| DistilBERT | 0.92 | 0.90 | 0.91 | 0.96 | 0.96 | 0.96 |
| MPNet | 0.93 | 0.88 | 0.90 | 0.96 | 0.96 | 0.96 |
| RoBERTa | 0.90 | 0.90 | 0.90 | 0.96 | 0.96 | 0.96 |
| ALBERT | 0.90 | 0.88 | 0.89 | 0.95 | 0.95 | 0.95 |

**Table 5**
Classification report comparison: inference time and accuracy.

| Model | Inference time (s) | Accuracy |
|---|---|---|
| Proposed model | 11.06 | 0.98 |
| DeBERTa | 18.21 | 0.96 |
| BERT | 11.22 | 0.97 |
| DistilBERT | 5.98 | 0.96 |
| MPNet | 13.19 | 0.96 |
| RoBERTa | 11.19 | 0.96 |
| ALBERT | 12.14 | 0.95 |

**Table 6**
Standard deviation report: macro and weighted scores.

| Model | Macro | | | Weighted | | |
|---|---|---|---|---|---|---|
| | Precision | Recall | F1 | Precision | Recall | F1 |
| Proposed model | 0.01 | 0.01 | 0.01 | 0.00 | 0.00 | 0.00 |
| DeBERTa | 0.01 | 0.01 | 0.01 | 0.00 | 0.01 | 0.01 |
| BERT | 0.02 | 0.01 | 0.00 | 0.00 | 0.00 | 0.00 |
| DistilBERT | 0.01 | 0.02 | 0.01 | 0.00 | 0.00 | 0.00 |
| MPNet | 0.03 | 0.03 | 0.01 | 0.00 | 0.00 | 0.00 |
| RoBERTa | 0.03 | 0.02 | 0.01 | 0.00 | 0.01 | 0.01 |
| ALBERT | 0.02 | 0.02 | 0.01 | 0.00 | 0.00 | 0.00 |

**Table 7**
Standard deviation report: inference time and accuracy.

| Model | Inference time (s) | Accuracy |
|---|---|---|
| Proposed model | 0.80 | 0.00 |
| DeBERTa | 1.31 | 0.01 |
| BERT | 2.15 | 0.00 |
| DistilBERT | 0.55 | 0.00 |
| MPNet | 0.29 | 0.00 |
| RoBERTa | 1.12 | 0.01 |
| ALBERT | 0.24 | 0.00 |

**Table 8**
Classification report comparison: Granular class-wise.

| Model | Tip-over | | | Non-tip-over | | |
|---|---|---|---|---|---|---|
| | Precision | Recall | F1 | Precision | Recall | F1 |
| Proposed model | 0.94 | 0.92 | 0.93 | 0.99 | 0.99 | 0.99 |
| DeBERTa | 0.84 | 0.86 | 0.85 | 0.98 | 0.97 | 0.98 |
| BERT | 0.88 | 0.82 | 0.85 | 0.97 | 0.99 | 0.98 |
| DistilBERT | 0.87 | 0.81 | 0.84 | 0.97 | 0.98 | 0.98 |
| MPNet | 0.89 | 0.78 | 0.83 | 0.97 | 0.98 | 0.98 |
| RoBERTa | 0.82 | 0.84 | 0.83 | 0.98 | 0.97 | 0.98 |
| ALBERT | 0.83 | 0.78 | 0.80 | 0.97 | 0.98 | 0.97 |

**Table 9**
Standard deviation report: Granular class-wise.

| Model | Tip-over | | | Non-tip-over | | |
|---|---|---|---|---|---|---|
| | Precision | Recall | F1 | Precision | Recall | F1 |
| Proposed model | 0.03 | 0.02 | 0.01 | 0.00 | 0.01 | 0.00 |
| DeBERTa | 0.03 | 0.01 | 0.02 | 0.00 | 0.01 | 0.00 |
| BERT | 0.04 | 0.03 | 0.00 | 0.00 | 0.01 | 0.00 |
| DistilBERT | 0.03 | 0.04 | 0.01 | 0.01 | 0.01 | 0.00 |
| MPNet | 0.07 | 0.07 | 0.02 | 0.01 | 0.01 | 0.00 |
| RoBERTa | 0.06 | 0.05 | 0.02 | 0.01 | 0.01 | 0.01 |
| ALBERT | 0.05 | 0.06 | 0.02 | 0.01 | 0.01 | 0.00 |

diagram of inference time in Fig. 10 also statistically confirmed that our proposed model took the same time as other BERT variations when tasked with predicting tip-over incidents. While DistilBERT was the fastest language model, and DeBERTa was the slowest, our proposed model still was the best model with similar inference time, yet better results for our furniture tip-over classification task.

## 5. Conclusion

Manually reviewing and classifying individual incident reports requires tremendous effort and time. In this paper, we propose the application of machine learning to reduce human efforts with better results. In our experiment, we found that using default language models alone achieved good results on classification tasks, even though the

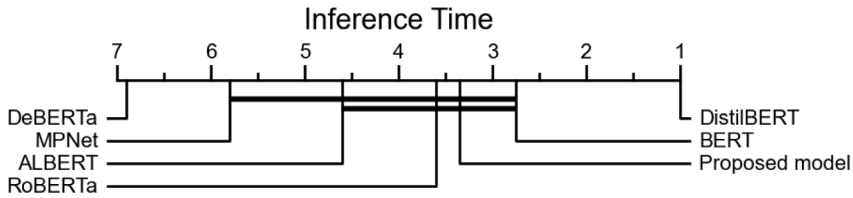



**Fig. 10.** Critical diagram of inference time.

data set contained abbreviations and the target classes were imbalanced. Optimizing a default model by adding a normalization layer helped the model reach at least 4% improvement on this specific task. When analyzing free text incident descriptions, especially when it comes to large datasets, a pretrained language model with layer normalization can help streamline workflow and enable greater efficiency for classification tasks.

Our proposed model not only offers great performance on this specific task – furniture tip-over classification – it also shows promise for applying this technique on other datasets when it comes to classification tasks. This technique may also be utilized in multiclass problems with additional efforts, such as modifying model architecture and tuning specific parameters for the best model performance.

## CRediT authorship contribution statement

**Ben Rodrawangpai:** Conceptualization, Methodology, Data curation, Writing – review & editing, Software, Visualization. **Witawat Daungjaiboon:** Investigation, Methodology, Conceptualization, Validation, Writing – original draft.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## References

Aggarwal, Charu C., & Zhai, ChengXiang (2012). A survey of text classification algorithms. In *Mining text data* (pp. 163–222). Boston, MA: Springer US, http://dx.doi.org/10.1007/978-1-4614-3223-4_6.

Ba, Jimmy Lei, Kiros, Jamie Ryan, & Hinton, Geoffrey E. (2016). Layer normalization. http://arxiv.org/abs/1607.06450 [arXiv:1607.06450].

Buitinck, Lars, Louppe, Gilles, Blondel, Mathieu, Pedregosa, Fabian, Mueller, Andreas, Grisel, Olivier, Niculae, Vlad, Prettenhofer, Peter, Gramfort, Alexandre, Grobler, Jaques, Layton, Robert, VanderPlas, Jake, Joly, Arnaud, Holt, Brian, & Varoquaux, Gaël (2013). API design for machine learning software: experiences from the scikit-learn project. In *ECML PKDD workshop: languages for data mining and machine learning* (pp. 108–122).

CPSC (2019). NEISS coding manual. URL: https://www.cpsc.gov/s3fs-public/2019_NEISS_Coding_Manual.pdf.

CPSC (2020). NEISS highlights, data and query builder. URL: https://www.cpsc.gov/cgibin/NEISSQuery/home.aspx.

Devlin, Jacob, Chang, Ming-Wei, Lee, Kenton, & Toutanova, Kristina (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. http://arxiv.org/abs/1810.04805 [arXiv:1810.04805].

Fawaz, Hassan Ismail, Forestier, Germain, Weber, Jonathan, Idoumghar, Lhassane, & Muller, Pierre-Alain (2019). Deep learning for time series classification: A review. *Data Mining and Knowledge Discovery*, *33*(4), 917–963. http://dx.doi.org/10.1007/s10618-019-00619-1.

He, Pengcheng, Liu, Xiaodong, Gao, Jianfeng, & Chen, Weizhu (2020). DeBERTa: Decoding-enhanced BERT with disentangled attention. http://dx.doi.org/10.48550/ARXIV.2006.03654, arXiv, URL: https://arxiv.org/abs/2006.03654.

Kingma, Diederik P., & Ba, Jimmy (2017). Adam: A method for stochastic optimization. http://arxiv.org/abs/1412.6980 [arXiv:1412.6980].

Kowsari, Meimandi, Jafari, Mendu, Heidarysafa, & Brown, Barnes (2019). Text classification algorithms: A survey. *Information*, *10*(4), 150. http://dx.doi.org/10.3390/info10040150.

Lan, Zhenzhong, Chen, Mingda, Goodman, Sebastian, Gimpel, Kevin, Sharma, Piyush, & Soricut, Radu (2019). ALBERT: A lite BERT for self-supervised learning of language representations. http://dx.doi.org/10.48550/ARXIV.1909.11942, URL: https://arxiv.org/abs/1909.11942.

Lee, Jieh-Sheng, & Hsiang, Jieh (2019). PatentBERT: Patent classification with fine-tuning a pre-trained BERT model. http://dx.doi.org/10.48550/ARXIV.1906.02124, arXiv, URL: https://arxiv.org/abs/1906.02124.

Li, Shaobo, Hu, Jie, Cui, Yuxin, & Hu, Jianjun (2018). DeepPatent: Patent classification with convolutional neural networks and word embedding. *Scientometrics*, *117*(2), 721–744. http://dx.doi.org/10.1007/s11192-018-2905-5.

Liu, Yinhan, Ott, Myle, Goyal, Naman, Du, Jingfei, Joshi, Mandar, Chen, Danqi, Levy, Omer, Lewis, Mike, Zettlemoyer, Luke, & Stoyanov, Veselin (2019). RoBERTa: A robustly optimized BERT pretraining approach. http://arxiv.org/abs/1907.11692 [arXiv:1907.11692].

Lu, Chang, Badeti, Jaahnavi, Mehan, Tracy J., Zhu, Motao, & Smith, Gary A. (2021). Furniture and television tip-over injuries to children treated in United States emergency departments. *Injury Epidemiology*, *8*(1), 53.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Ramos, Juan (2003). Using TF-IDF to determine word relevance in document queries.

Rish, Irina (2001). An empirical study of the naïve Bayes classifier. In *IJCAI 2001 Work Empir Methods Artif Intell, vol. 3*.

Sanh, Victor, Debut, Lysandre, Chaumond, Julien, & Wolf, Thomas (2020). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. http://arxiv.org/abs/1910.01108 [arXiv:1910.01108].

Song, Kaitao, Tan, Xu, Qin, Tao, Lu, Jianfeng, & Liu, Tie-Yan (2020). MPNet: Masked and permuted pre-training for language understanding. http://dx.doi.org/10.48550/ARXIV.2004.09297, URL: https://arxiv.org/abs/2004.09297.

Srivastava, Nitish, Hinton, Geoffrey, Krizhevsky, Alex, Sutskever, Ilya, & Salakhutdinov, Ruslan (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, *15*(56), 1929–1958, URL: http://jmlr.org/papers/v15/srivastava14a.html.

Suchy, Adam (2020). *Product instability or tip-over injuries and fatalities associated with televisions, furniture, and appliances: 2020 report*. US Consumer Product Safety Commission, URL: https://www.cpsc.gov/s3fs-public/2020_Tip_Over_Report.pdf.

Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, & Polosukhin, Illia (2017). Attention is all you need. http://dx.doi.org/10.48550/ARXIV.1706.03762, arXiv, URL: https://arxiv.org/abs/1706.03762.

Wolf, Thomas, Debut, Lysandre, Sanh, Victor, Chaumond, Julien, Delangue, Clement, Moi, Anthony, Cistac, Pierric, Rault, Tim, Louf, Rémi, Funtowicz, Morgan, Davison, Joe, Shleifer, Sam, von Platen, Patrick, Ma, Clara, Jernite, Yacine, Plu, Julien, Xu, Canwen, Scao, Teven Le, Gugger, Sylvain, .... Rush, Alexander M. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations* (pp. 38–45). Online: Association for Computational Linguistics, URL: https://www.aclweb.org/anthology/2020.emnlp-demos.6.

Xu, Jingjing, Sun, Xu, Zhang, Zhiyuan, Zhao, Guangxiang, & Lin, Junyang (2019). Understanding and improving layer normalization. http://arxiv.org/abs/1911.07013 [arXiv:1911.07013].